

Documentacion Tecnica

Sistema Experto de Cotizacion - Maderas Gerardo

Munoz y Girata — Inteligencia Artificial II — Los Libertadores, 2026

1. Introduccion

1.1 Descripcion del dominio

Maderas Gerardo es un taller de carpinteria artesanal ubicado en Bogota, Colombia, en operacion desde 1998. Fabrica muebles a medida: puertas, closets, cocinas integrales, camas, mesas y estantes, trabajando con MDF, triplex, madera solida y pino. El proceso de cotizacion dependia exclusivamente de la experiencia del carpintero, lo que generaba inconsistencias. Este sistema experto formaliza ese conocimiento en 40 reglas de produccion reproducibles.

1.2 Objetivo del sistema

Construir un sistema experto con motor de inferencia por encadenamiento hacia adelante que reciba las caracteristicas del mueble deseado (tipo, material, color, acabado, extras) y produzca una cotizacion con precio estimado, tiempo de entrega, desglose de costos y explicacion de las reglas aplicadas mediante la funcion porque().

2. Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor de tres capas: capa de presentacion (React), capa de logica (FastAPI) y capa de conocimiento (motor de inferencia). La comunicacion entre frontend y backend se realiza mediante HTTP/REST con intercambio de datos en formato JSON.

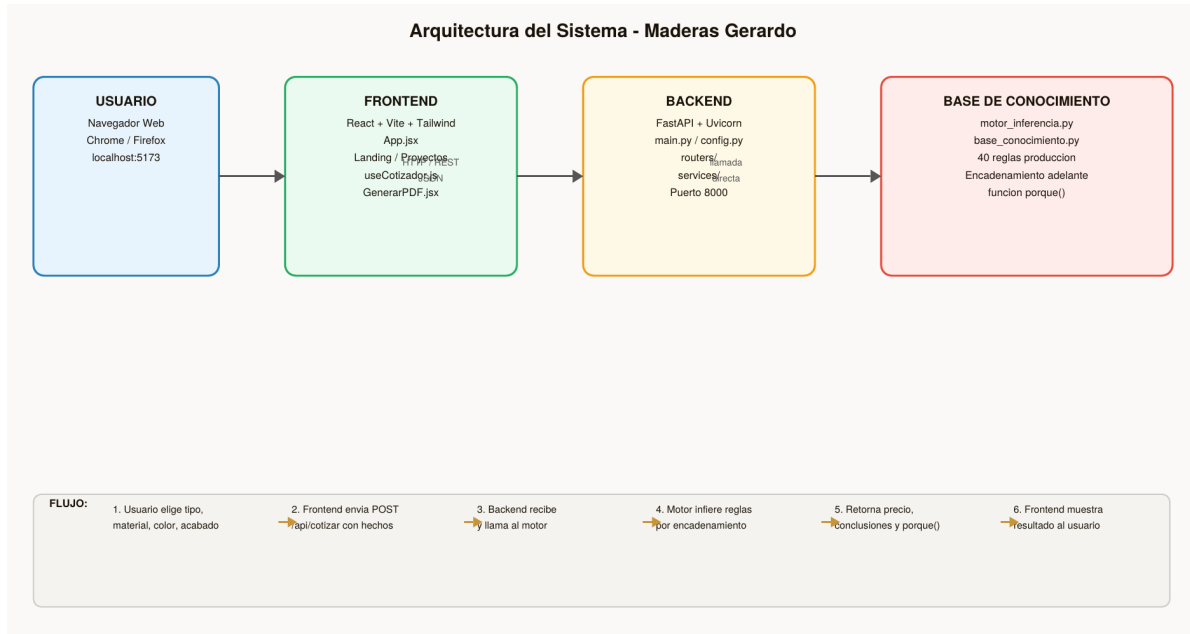


Figura 1. Diagrama de arquitectura del sistema con flujo de datos de una cotización completa.

2.1 Componentes principales

Frontend (React + Vite + Tailwind): interfaz de usuario con cotizador paso a paso, visualizador SVG del mueble en tiempo real, sección de resultados con desglose y panel de explicación técnica del motor.

Backend (FastAPI + Uvicorn): API REST que expone el endpoint POST /api/cotizar. Recibe los hechos del usuario, invoca el motor de inferencia y retorna las conclusiones en JSON.

Base de conocimiento: módulo Python con las 40 reglas de producción (base_conocimiento.py) y el algoritmo de encadenamiento hacia adelante (motor_inferencia.py).

2.2 Estructura de directorios

```
src/backend/main.py -- punto de entrada FastAPI
src/backend/config.py -- configuracion centralizada
src/backend/routers/ -- endpoints REST
src/backend/services/ -- logica de negocio
src/backend/knowledge/base_conocimiento.py -- 40 reglas
src/backend/knowledge/motor_inferencia.py -- algoritmo
src/frontend/src/App.jsx -- componente raiz
src/frontend/src/hooks/useCotizador.js -- estado del cotizador
```

3. Formalizacion en logica de predicados

3.1 Predicados del dominio

El conocimiento del taller se formaliza con los siguientes predicados:

Predicado	Descripcion	Categoria
es_puerta(x)	x es una puerta	Tipo
es_closet(x)	x es un closet	Tipo
es_cocina(x)	x es una cocina integral	Tipo
es_cama(x)	x es una cama	Tipo
es_mesa(x)	x es una mesa	Tipo
es_estante(x)	x es un estante	Tipo
material_mdf(x)	x usa MDF como material	Material
material_triplex(x)	x usa triplex	Material
material_solido(x)	x usa madera solida	Material
material_pino(x)	x usa pino	Material
acabado_lacado(x)	x tiene acabado lacado	Acabado
acabado_barniz(x)	x tiene barniz	Acabado
acabado_pintado(x)	x tiene pintura	Acabado
requiere_instalacion(x)	requiere instalacion en sitio	Extra
cotizacion(x, P, T)	precio P y tiempo T para x	Conclusion

3.2 Ejemplo de formalizacion

Regla R01 en logica de predicados:

```
FORALL x: es_puerta(x) AND material_mdf(x) => cotizacion(x, menor_80, 3_5_dias) [CF=0.90]
```

Regla R05 en logica de predicados:

```
FORALL x: es_closet(x) AND material_solido(x) AND acabado_lacado(x) => cotizacion(x, menor_500, 15_20_dias) [CF=0.95]
```

4. Base de conocimiento - Reglas de produccion

El sistema implementa 40 reglas organizadas por tipo de mueble, material, acabado y extras. A continuacion se presenta una muestra representativa de 16 reglas:

ID	Condicion (SI)	Conclusion (ENTONCES)	CF
R01	es_puerta AND material_mdf	Puerta en MDF - precio <\$80	0.90
R02	es_puerta AND material_triplex	Puerta triplex - precio <\$150	0.93
R03	es_puerta AND material_solido	Puerta solida - precio <\$280	0.96
R04	es_closet AND material_mdf	Closet MDF - precio <\$200	0.88
R05	es_closet AND material_solido AND lac.	Closet premium - precio <\$500	0.95
R06	es_cocina AND material_mdf	Cocina MDF - precio <\$350	0.85
R07	es_cocina AND material_solido	Cocina premium - precio <\$900	0.92
R08	es_cama AND material_mdf	Cama MDF - precio <\$180	0.87
R09	es_cama AND material_solido	Cama solida - precio <\$420	0.94
R10	es_mesa AND material_pino	Mesa pino - precio <\$120	0.89
R11	es_estante AND material_mdf	Estante MDF - precio <\$90	0.86
R12	es_estante AND material_solido	Estante solido - precio <\$240	0.91
R20	acabado_lacado	Recargo lacado +30%	1.00
R21	acabado_barniz	Recargo barniz +20%	1.00
R30	requiere_instalacion	Recargo instalacion en sitio	1.00
R40	material_solido AND acabado_lacado	Advertencia: combinacion premium	0.98

Nota: La tabla muestra 16 de las 40 reglas. El archivo completo esta en `src/backend/knowledge/base_conocimiento.py`

5. Algoritmo del motor de inferencia

5.1 Encadenamiento hacia adelante

El motor implementa forward chaining partiendo de hechos conocidos y derivando conclusiones hasta que no se pueden inferir nuevos hechos:

1. Recibir hechos iniciales: tipo, material, color, acabado, extras
2. Inicializar agenda con las 40 reglas de la base de conocimiento
3. Para cada regla: verificar si TODAS sus condiciones estan en los hechos actuales
4. Si se cumplen: agregar la conclusion a los hechos derivados y registrar la regla
5. Repetir desde paso 3 hasta que no se agreguen nuevos hechos (punto fijo)
6. Calcular precio final integrando los CF de las reglas aplicadas

5.2 Funcion porque()

El motor registra cada regla que se dispara. La funcion porque() retorna esa lista con condicion, conclusion y CF de cada regla. Esta informacion alimenta el panel "Explicacion Tecnica - Motor de Inferencia" visible en la interfaz (ver Figura 5 del manual de usuario).

5.3 Casos sin conclusion

Si ningun conjunto de condiciones se satisface completamente, el motor retorna precio nulo con mensaje explicativo. Esto evita cotizaciones con valores incorrectos cuando los datos son insuficientes.

6. Validacion

Se documentaron y ejecutaron 5 casos de prueba:

#	Nombre	Entrada	Resultado esperado	Estado
1	Basico	Puerta + MDF + Natural	Precio <\$80, 1 regla	PASS
2	Complejo	Closet + Solida + Lacado + Instalacion	Precio <\$500, 3+ reglas	PASS
3	Sin datos	Solo tipo sin material	Mensaje sin datos	PASS
4	Advertencia	Cocina + Solida + Lacado	Precio + advertencia	PASS
5	Borde	Cama + Triplex + Barniz + todos extras	Precio multiples reglas	PASS

Todos los casos pasaron satisfactoriamente. Detalles completos en tests/casos_prueba.md

7. Decisiones de diseño

7.1 FastAPI sobre Flask

FastAPI genera documentación automática (Swagger en /docs), tiene validación de tipos con Pydantic integrada y es asíncrono. Para un sistema experto que puede crecer, estas características reducen el tiempo de depuración.

7.2 Encadenamiento hacia adelante

El dominio es una cotización: el usuario da hechos conocidos y el sistema deriva el precio. Este flujo es naturalmente compatible con forward chaining. Backward chaining sería adecuado para verificar si un precio es correcto, no para calcularlo.

7.3 Separación de capas

config.py centraliza parámetros, models/schemas.py define estructuras, routers/ expone endpoints, services/ maneja lógica, knowledge/ contiene el motor. Esta separación permite modificar las reglas sin tocar la API.

7.4 Factores de certeza

Cada regla tiene un CF entre 0 y 1 que refleja la confianza del experto en esa regla. Los CF se combinan multiplicativamente al encadenar reglas, permitiendo manejar incertidumbre sin modelo probabilístico completo.

8. Conclusiones

8.1 Logros alcanzados

Se construyo un sistema experto funcional con 40 reglas de produccion, motor de inferencia con encadenamiento hacia adelante, interfaz web profesional con visualizador en tiempo real, generacion de PDF, contacto por WhatsApp y explicacion del razonamiento mediante porque(). El sistema esta basado en un dominio real y produce cotizaciones consistentes y auditables.

8.2 Aprendizajes

La formalizacion del conocimiento experto requiere iteracion con el experto del dominio. La separacion en capas facilita el mantenimiento. La validacion con casos reales es fundamental para detectar reglas faltantes o mal formuladas.

8.3 Mejoras futuras

Configurador visual 3D, aprendizaje automatico para ajustar CF con retroalimentacion real, integracion con inventario del taller, y despliegue en servidor cloud para acceso publico.